SS - CheatSheet

GTU BE CE | Semester 5 | BE05000261 | Syllabus 2024-25

> **Compiled BY** : Manthan A. D. [in]

1. Exam priorities and study order

Priority	Topics to prepare first
Very high	Left recursion removal, Symbol Table and Intermediate Code, LL(1) parsing table
High	Assembler data structures, macro tables, macro expansion, relocation, loaders
High	Code optimization, triples/quadruples, JVM, interpreter types
Medium	FIRST and FOLLOW, shift-reduce parsing, grammar types, binding, debugging
New syllabus	Chapter 9: language features, storage allocation, scope, extensibility

Syllabus at a glance

Chapter	Topic	Weightage in syllabus
1	Overview of System Software	5%
2	Language Processors	10%
3	Assemblers	15%
4	Macro and Macro Processors	15%
5	Linkers and Loaders	15%
6	Scanning and Parsing	10%
7	Compilers	10%
8	Interpreters and Debuggers	10%
9	Programming Languages	10%

Chapter 1. Overview of System Software

A. Essential terminology

Term	Definition
Software	A set of programs and related instructions used to operate a computer.
System Software	Software that manages computer resources or provides an environment for developing and executing programs.
Application Software	Software designed to perform user tasks, such as browsers or text editors.
System Programming	Developing software that manages hardware, system resources, or program execution.
Machine Structure	The organization of the CPU, memory, input/output devices, and their interconnections.

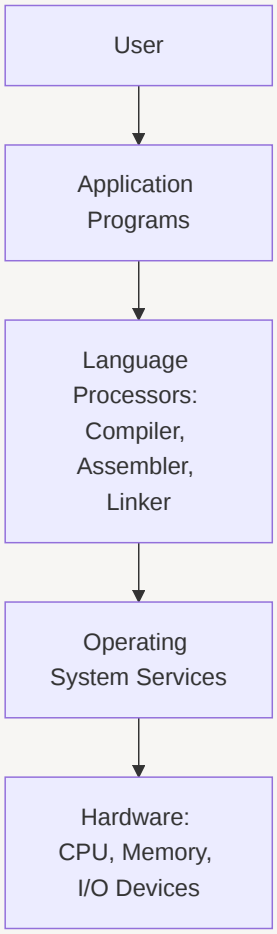
Term	Definition
Interface	A defined boundary through which two components communicate.
Address Space	The set or range of memory addresses available to a program or process.
System Software Hierarchy	The layered relationship between hardware, system services, language-processing tools, and applications.

B. Types and levels of software

Typical components of system software include:

- **Operating system:** Manages processes, memory, files, and devices.
- **Assembler:** Converts assembly language into machine code.
- **Compiler:** Translates a source program into another representation, commonly machine code or assembly.
- **Interpreter:** Executes a program or intermediate representation by interpreting its instructions.
- **Linker:** Combines object modules and resolves external references.
- **Loader:** Places an executable program into memory for execution.
- **Macro processor:** Expands macro calls into their corresponding instruction sequences.

System software hierarchy

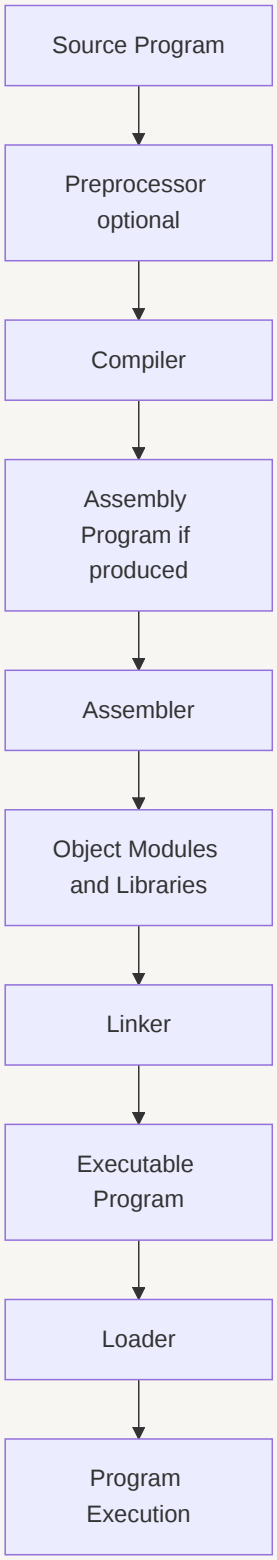



```
       User
        |
 Application Programs
        |
Language Processors
Compiler, Assembler, Linker
        |
Operating System Services
        |
Hardware
CPU, Memory, I/O Devices
```

This is a conceptual hierarchy. The precise placement of components can vary by system.

C. Life cycle of a source program

Important diagram. Understand each stage and its output.



Some compilers generate object code directly, so a separate assembler may not be required.

Stage	Main task	Output
Preprocessor	Expands macros and handles source directives	Modified source
Compiler	Translates high-level language	Assembly or object code
Assembler	Converts assembly instructions	Object module
Linker	Combines modules and resolves external symbols	Executable or linked module
Loader	Places program into memory and prepares execution	Loaded program

Remember: Linking is about combining and resolving; loading is about placing a program into memory.

D. Different views of a program

- **User-centric view:** Focuses on what the program does and the problem it solves.
- **System-centric view:** Focuses on instructions, memory, resources, interfaces, and execution.
- **System software development:** Involves creating tools and services that support program translation, loading, and execution.

E. Three important gaps

Term	Meaning
Semantic Gap	Difference between the concepts used to express a problem and the low-level operations supported by a machine.
Specification Gap	Difference between describing what a system should do and expressing that requirement in a form suitable for implementation.
Execution Gap	Difference between expressing an operation in a programming language and the actual steps required to execute it on hardware.

These terms describe different aspects of the distance between a problem, its specification, its programming-language representation, and machine execution.

Chapter 1 quick revision: System software, software hierarchy, machine structure, address space, user-centric versus system-centric views, and source-program life cycle.

Chapter 2. Overview of Language Processors

A. Essential terminology

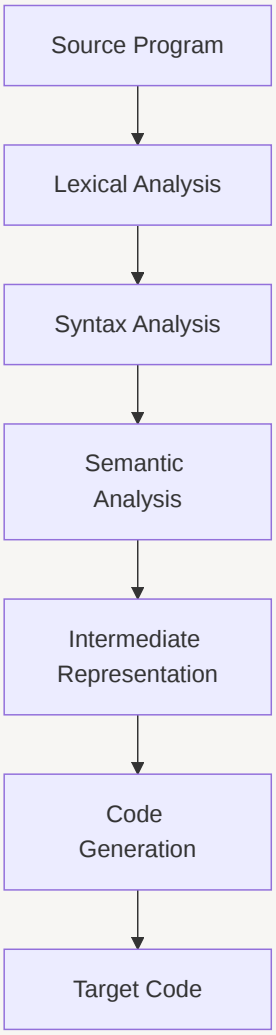
Term	Definition
Programming Language	A formal language used to express computations and instructions.
Language Processor	A program that translates or interprets programs written in a programming language.
Translator	Converts a program from one language or representation to another.
Token	A category of meaningful lexical units, such as identifiers, numbers, and operators.
Lexeme	The actual character sequence matched to a token, such as `count` for an identifier token.
Symbol Table (SYMTAB)	A data structure that stores information about identifiers, including their names, addresses, types, or attributes.
Intermediate Representation (IR)	A representation between source code and final target code.

Term	Definition
Language Migrator	A tool or process that assists in moving a program between languages, platforms, or environments.

B. Main language processors

Processor	Function
Assembler	Assembly language to machine/object code
Compiler	Translates the source program into another representation
Interpreter	Executes instructions through interpretation
Preprocessor	Performs source-level transformations before compilation
Macro processor	Expands macro definitions and calls
Linker	Combines modules and resolves external symbols
Loader	Places a program into memory for execution

C. Fundamental language-processing activities



This is a simplified compiler-oriented view. Actual processing stages vary by language and toolchain.

- **Lexical analysis:** Groups characters into tokens.
- **Syntax analysis:** Checks whether token sequences follow grammar rules.
- **Semantic analysis:** Checks meaning-related constraints, such as type compatibility.
- **Intermediate-code generation:** Produces an intermediate representation.
- **Code generation:** Produces target instructions.
- **Optimization:** Improves efficiency without changing required program behavior.

D. Symbol table operations

A symbol table commonly supports:

1. **Insert:** Add a newly encountered symbol.
2. **Search/Lookup:** Find a symbol by name.

3. **Update:** Modify its attributes.
4. **Retrieve:** Obtain information such as address, type, or scope.

Common implementation data structures

Structure	Advantage	Limitation
Linear list	Simple to implement	Search may take O(n) time
Binary search tree	Ordered searching; potentially O(log n) when balanced	Can degrade if unbalanced
Hash table	Average-case lookup is often O(1)	Collisions require handling
Balanced search tree	Predictable O(log n) search and insertion	More complex to maintain

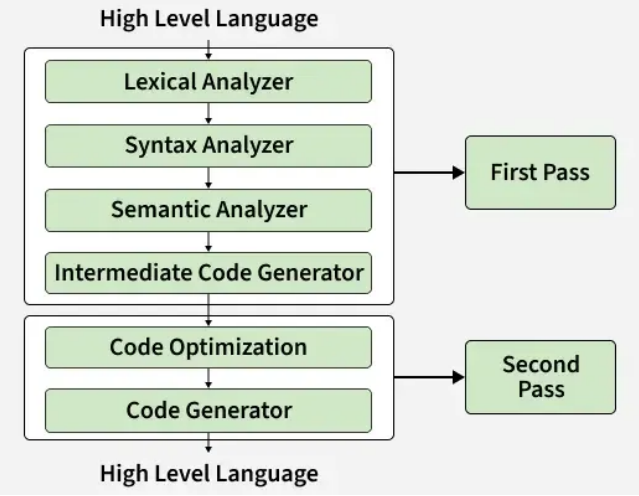
Keywords: Token, lexeme, translator, symbol table, intermediate representation, lexical analysis.

Chapter 3. Assemblers

Highest-priority chapter

A. Essential terminology

Term	Definition
Assembly Language	A low-level language using mnemonic instructions and symbolic operands.
Assembler	A translator that converts assembly language into machine/object code.
Mnemonic	A readable instruction name, such as `ADD` or `MOVER`.
Label	A symbolic name assigned to a program address.
Location Counter (LC)	A counter that tracks the address assigned to the next assembled statement.
Opcode	The machine-operation code associated with an instruction.
Operand	A value, register, address, or symbol on which an instruction operates.
Forward Reference	Use of a symbol before its definition appears.
Backpatching	Filling in previously unresolved values or addresses after the required information becomes available.



B. Common assembly instructions

The following are common **SAL-style educational examples**. Exact syntax, mnemonics, and opcode assignments depend on the assembler.

Instruction	General meaning
`MOVER`	Load a register from a memory operand
`MOVEM`	Store a register value into memory
`ADD`	Add an operand to a register
`SUB`	Subtract an operand
`MULT`	Multiply
`DIV`	Divide
`COMP`	Compare operands
`BC`	Conditional branch
`READ`	Read input
`PRINT`	Produce output
`STOP`	Stop execution

Registers such as `AREG`, `BREG`, `CREG`, and `DREG` are used in educational assembly examples.

C. Important assembler directives

An **assembler directive** controls the assembly process rather than directly performing a normal CPU operation.

Directive	Meaning	Main purpose
`START`	Specify the starting address	Initialize LC
`END`	Mark the end of the source program	Finish assembly and process pending literals where applicable
`ORIGIN`	Set LC to a specified address or expression	Change the assembly location
`EQU`	Associate a symbol with a value or expression	Define symbolic constants or equivalent values
`LTORG`	Literal origin/pool directive	Assign addresses to pending literals
`DS`	Define Storage	Reserve memory
`DC`	Define Constant	Allocate storage and initialize a value

Difference between `DS` and `DC`

- `DS 1`: Reserve one storage unit without specifying an initial constant.
- `DC 5`: Allocate one storage unit initialized with the constant `5`.

The exact storage unit depends on the target architecture and assembler.

D. Assembler data structures

These tables are especially important in theory questions.

Table	Full name	Purpose
OPTAB	Operation Code Table	Stores mnemonics, opcode information, and instruction lengths

Table	Full name	Purpose
SYMTAB	Symbol Table	Stores symbol names, addresses, and associated attributes
LITTAB	Literal Table	Stores literals and their assigned addresses
POOLTAB	Pool Table	Identifies the starting index of each literal pool

Other tables may be used by a particular assembler implementation.

Literal versus symbol

- **Symbol:** A named program entity, such as `RESULT`.
- **Literal:** A constant written directly in the instruction, such as `='5'`.

E. One-pass versus two-pass assembler

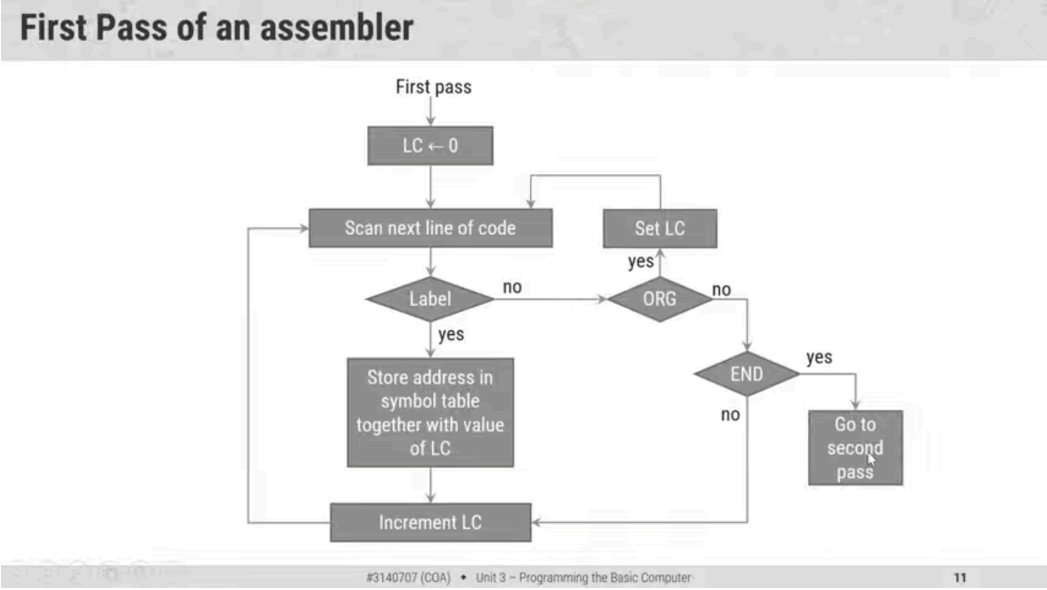
Feature	One-pass assembler	Two-pass assembler
Main process	Processes the program in one principal scan	Processes the program in two principal passes
Forward references	May need fix-up chains or backpatching	Usually resolved using information collected in Pass 1
Processing	Generally avoids a separate initial full pass	Uses Pass 1 for addresses/tables and Pass 2 for code generation
Design	Can be harder when references are unresolved	Easier to organize for forward-reference resolution
Typical use	Suitable when the assembler's design supports its restrictions	Common educational design for general assembly processing

A one-pass assembler can handle forward references with appropriate mechanisms. It is not correct to say that every one-pass assembler is unable to handle them.

F. Two-pass assembler algorithm

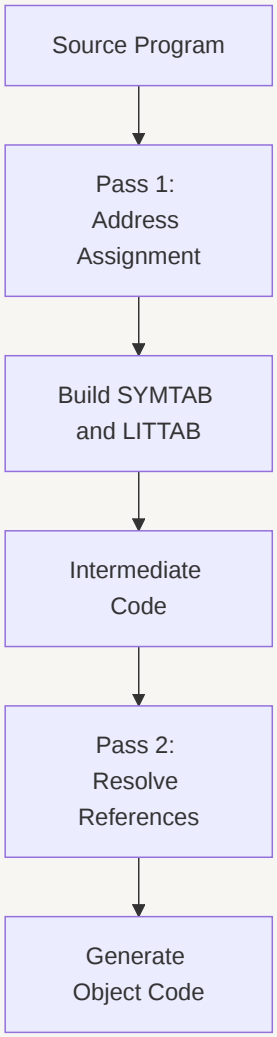
Pass 1

- Initialize LC.
- Process labels, instructions, and assembler directives.
- Assign addresses to instructions and data.
- Build SYMTAB and LITTAB.
- Record literals and handle literal pools.
- Produce intermediate code or equivalent records.



Pass 2

- Read the intermediate representation.
- Consult OPTAB and SYMTAB.
- Assign literal addresses using LITTAB.
- Resolve forward references.
- Generate object/machine code.



In Short,



G. Worked example: LC, SYMTAB and LITTAB

Consider this SAL-style program:

```
        START 100
        MOVER AREG, ='5'
        ADD   AREG, ONE
        MOVEM AREG, RESULT
ONE     DC    1
RESULT  DS    1
```

```
        LTORG
        END
```

Assume each instruction and allocated storage unit consumes one address unit.

Statement	Address / action	LC after processing
`START 100`	Initialize LC	100
`MOVER AREG, ='5'`	Instruction at 100	101
`ADD AREG, ONE`	Instruction at 101	102
`MOVEM AREG, RESULT`	Instruction at 102	103
`ONE DC 1`	`ONE` defined at 103	104
`RESULT DS 1`	`RESULT` defined at 104	105
`LTORG`	Place literal `='5'` at 105	106
`END`	End assembly	106

Resulting tables:

SYMTAB

Symbol	Address
`ONE`	103
`RESULT`	104

LITTAB

Literal	Address
`='5'`	105

POOLTAB: The first literal pool begins at the index of the first literal in LITTAB. In this example, that is index 1 under one-based indexing.

Important: The actual object-code bit patterns and opcode numbers cannot be determined from this example alone. They depend on the target instruction set and assembler tables.

H. `EQU`, `ORIGIN` and forward references

```
        START 200
A       DS    1
B       EQU   A+5
        ORIGIN A+3
C       DS    1
        END
```

Step-by-step:

- `A DS 1`: `A` gets address 200; LC becomes 201.
- `B EQU A+5`: `B` is assigned the value 205; no storage is allocated for `B`.
- `ORIGIN A+3`: LC becomes 203.
- `C DS 1`: `C` gets address 203; LC becomes 204.

`ORIGIN` can change the LC forward or backward. A poorly chosen value can cause storage overlaps, so use it carefully.

Forward reference example

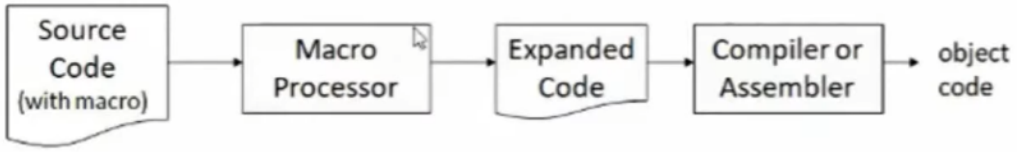
```
        START 100
        MOVER AREG, VALUE
        ...
VALUE   DC    7
        END
```

When the first instruction is processed, `VALUE` has not yet been defined. A two-pass assembler can resolve its address after Pass 1 creates the symbol information.

Chapter 3 quick revision: OPTAB, SYMTAB, LITTAB, POOLTAB, LC, Pass 1/Pass 2, `EQU`, `ORIGIN`, `LTORG`, forward references, and backpatching.

Chapter 4. Macro and Macro Processors

High-priority chapter



A. Essential terminology

Term	Definition
Macro	A named sequence of statements that can be expanded where it is called.
Macro Definition	The template containing the macro body and parameters.
Macro Call	A request to expand a defined macro.
Macro Expansion	Replacing a macro call with its expanded body.
Formal Parameter	A parameter defined in the macro definition.
Actual Parameter	A value supplied when the macro is called.
Nested Macro Call	A macro expansion that calls another macro.
Macro Processor	Software that stores and expands macro definitions and calls.

B. Macro definition and expansion

Example:

```
MACRO
INCR &X
```

```
    LOAD  &X
    ADD   =1
    STORE &X
MEND
```

Macro call:

```
INCR COUNT
```

Conceptual expansion:

```
    LOAD  COUNT
    ADD   =1
    STORE COUNT
```

`&X` is a formal parameter. `COUNT` is the actual parameter substituted during expansion.

This is an illustrative macro notation; the exact syntax depends on the macro processor.

C. Macro versus subroutine

Feature	Macro	Subroutine
Processing	Expanded by a macro processor	Called during program execution
Code size	May increase because the body is copied	Usually stores one shared routine
Call overhead	No normal runtime call/return overhead from the expansion itself	Typically has call/return overhead
Execution speed	Can be faster for small bodies, depending on generated code	Useful for reusable routines
Flexibility	Supports parameter substitution and expansion-time facilities	Supports runtime control and shared routines
Maintenance	A single definition can generate many instruction sequences	One routine can serve multiple calls

Remember: Macro expansion occurs during preprocessing/assembly; a subroutine call normally occurs at runtime.

D. Macro processor tables

Table	Function
MNT	Macro Name Table. Stores macro names and associated pointers/attributes.
MDT	Macro Definition Table. Stores macro body statements.
ALA	Argument List Array. Maps formal parameters to actual arguments during expansion.
PNTAB	Parameter Name Table. Associates parameter names with parameter positions.
KPDTAB	Keyword Parameter Default Table. Stores keyword parameters and their default values.
EVTAB	Expansion Variable Table. Maintains expansion-time variables and their values.
SSTAB	Sequence Symbol Table. Stores sequence symbols and their related positions for expansion-time branching.

Fields vary by implementation. When explaining a table in an exam, mention its purpose and key information stored in it.

E. Parameter types

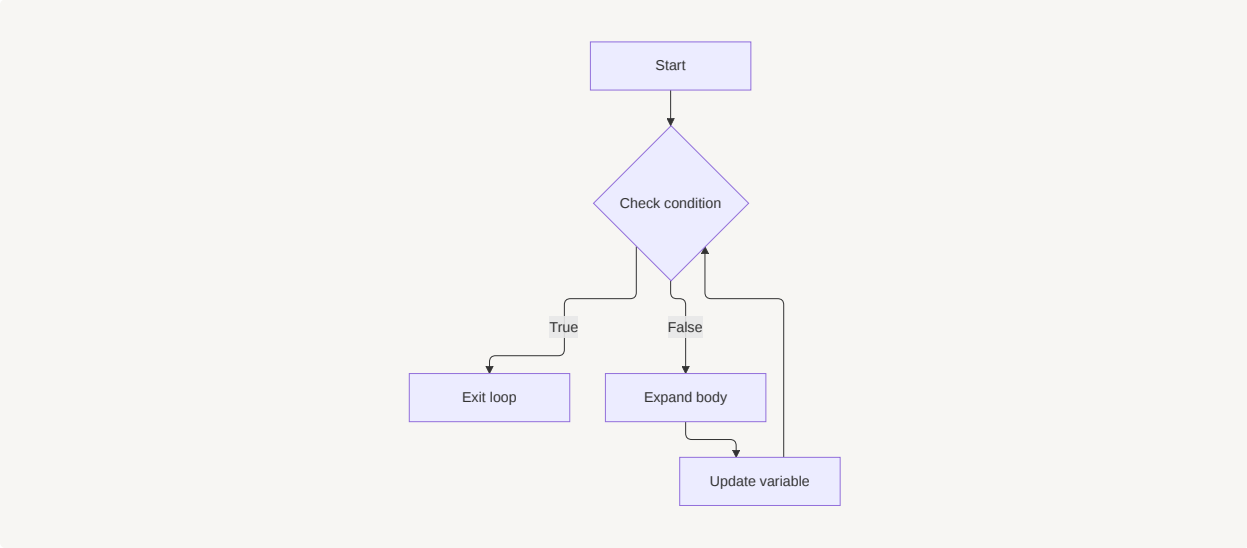
Type	Meaning	Example call
Positional	Argument matched by position	`SUM A, B`
Keyword	Argument identified by name	`SUM &X=A, &Y=B`
Default value	Argument receives a predefined value if omitted	`SUM &X=A, &Y=1`

Keyword/default syntax shown is conceptual, not a universal assembler syntax.

F. Expansion-time control statements

Statement	Purpose
AIF	Conditional branch during macro expansion
AGO	Unconditional branch to a sequence symbol during expansion
ANOP	No-operation statement that can serve as a sequence-symbol target
REPT	Repeats a group of statements a specified number of times
IRP	Iterates over a list of parameters or values, expanding the body for each item

Expansion-time loop concept



An `AIF` / `AGO` example generally uses sequence symbols to implement the condition and loop. Exact syntax depends on the processor.

G. One-pass and two-pass macro processors

- **One-pass macro processor:** Can define and expand macros in a single processing pass when the design and input ordering allow it.
- **Two-pass macro processor:** One pass processes macro definitions and builds tables; another expands macro calls.

- **Nested macro calls:** Require the processor to manage the active definition and parameter information correctly.
- **Lexical versus semantic expansion:** Lexical expansion focuses on textual substitution; more semantic approaches account for context or expansion-related meaning and rules.

Chapter 4 quick revision: Macro versus subroutine, definition/call/expansion, nested macros, parameter types, MNT, MDT, ALA, PNTAB, KPDTAB, EVTAB, SSTAB, AIF, AGO, ANOP, REPT and IRP.

Chapter 5. Linkers and Loaders

High-priority chapter

A. Essential terminology

Term	Definition
Object Module	A translated program unit containing machine code and related information.
Linker	Combines object modules and resolves external references.
Loader	Places an executable or loadable module in memory and prepares it for execution.
Relocation	Adjusting address-dependent values when a program is placed at a location different from its assumed origin.
External Reference	A reference to a symbol defined in another object module.
Load Address	The address in memory where a program or module is loaded.
Relocation Factor	Difference between the actual load origin and the origin assumed by the translated program, under a given relocation scheme.

B. Linker versus loader

Linker	Loader
Combines modules and libraries when required	Places the executable/module in memory
Resolves external symbols	Sets up the loaded program for execution
Performs or prepares linking/relocation work according to the format	Applies remaining loading/relocation work when needed
Produces a linked executable or loadable module in common systems	Initiates or prepares the program's execution

C. Relocation

Suppose a program was assembled assuming its origin is address 100, but it is loaded at address 500.

$$RF = \text{Actual Load Origin} - \text{Assumed Origin}$$

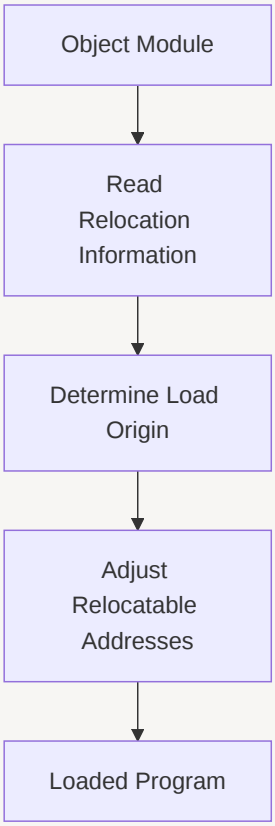
$$RF = 500 - 100 = 400$$

If an address-dependent value originally contains 125, its relocated value would be:

$$125 + 400 = 525$$

This simple addition applies to relocatable address fields under this example's assumptions. Real relocation formats may use different relocation records and field calculations.

Relocation process



D. Loading schemes

Scheme	Main idea
Compile-and-Go Loader	Translation is followed by placing the result in memory and executing it directly.
Absolute Loader	Loads code at the addresses specified in the object program; it does not perform general relocation.
Relocating Loader	Adjusts address-dependent information to support loading at a different location.
Direct Linking Loader	Loads object modules, resolves linking information, and performs the required relocation during loading.
Relocating Linking Loader	Combines linking and relocation work as the program is loaded.
Bootstrap Loader	A small initial loader used to bring in the software required to start the system.
Dynamic Linking	Defers at least some linking work until runtime or the point when a shared component is needed.

E. Self-relocating program

A **self-relocating program** determines where it is loaded and adjusts its own address-dependent information.

Key points:

- It can execute from different memory locations if designed correctly.
- It performs its own relocation calculations.
- It reduces dependence on an external relocating loader for the adjustments it handles.
- Its relocation code and assumptions must be designed correctly.

F. Overlay-structured programs

An **overlay** allows program parts to share the same region of memory because they are not needed simultaneously.

```
        Main Program
             |
      +--------+--------+
      |                 |
   Task A            Task B
      |                 |
      +---- Same Memory Region ----+
```

Task A and Task B occupy the shared region at different times. The main program and the selected overlay must be arranged so that simultaneously required components do not overwrite one another.

Benefit: Helps run programs when available memory cannot hold every component at once.

G. Address and size calculations

For a simple contiguous module:

$$\text{Module Size} = \text{Ending Address} - \text{Starting Address}$$

This assumes the ending address is an exclusive boundary. If the ending address points to the last occupied unit, use:

$$\text{Module Size} = \text{Last Address} - \text{Starting Address} + 1$$

Always check whether the question uses an inclusive or exclusive end address.

Chapter 5 quick revision: Relocation, relocation factor, absolute and relocating loaders, compile-and-go, direct linking, dynamic linking, self-relocating programs and overlays.

Chapter 6. Scanning and Parsing

Very high-priority chapter

A. Essential terminology

Term	Definition
Grammar	A formal set of production rules used to describe valid strings in a language.
Terminal	A symbol that appears in the generated input string, such as `+`, `*`, or `id`.
Non-terminal	A grammar variable that can be expanded using production rules, such as `E` or `T`.

Term	Definition
Production	A rewriting rule of a grammar.
Scanner/Lexer	Recognizes tokens from a stream of characters.
Parser	Checks the syntactic structure of a token sequence against a grammar.
FIRST	The set of terminals that can begin strings derived from a grammar symbol or sequence.
FOLLOW	The set of terminals that can appear immediately after a non-terminal in a valid sentential form.
Epsilon (ε)	The empty string.
LL(1)	A predictive parsing method that scans input Left to right, constructs a Leftmost derivation, and uses 1 lookahead token.
Handle	A substring whose reduction corresponds to one step in a reverse rightmost derivation.
Simple Phrase	In the usual simple-phrase definition, a phrase directly derived from a single non-terminal.

B. Types of grammar

Type	Name	General form
Type 0	Unrestricted grammar	General productions with a nonempty left-hand side containing a non-terminal
Type 1	Context-sensitive grammar	Length-nondecreasing productions, subject to the standard restrictions
Type 2	Context-free grammar	A → γ, with one non-terminal on the left
Type 3	Regular grammar	Right-linear or left-linear productions, such as A → aB or A → a

Each successive type is more restricted than the one above it in the Chomsky hierarchy.

C. Left recursion removal

Very important numerical question.

Immediate left recursion exists when a production begins with its own non-terminal:

$$A \to A\alpha \mid \beta$$

Standard transformation:

$$A \to \beta A'$$

$$A' \to \alpha A' \mid \varepsilon$$

For multiple alternatives:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2$$

Transform to:

$$A \to \beta_1 A' \mid \beta_2 A'$$

$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon$$

Here, the β alternatives must not themselves start with A, and the usual transformation presumes a suitable non-left-recursive alternative.

Example

Original:

$$E \to E + T \mid T$$

After removing left recursion:

$$E \to TE'$$

$$E' \to +TE' \mid \varepsilon$$

Method: Separate the recursive suffix from the non-recursive alternatives. Put the non-recursive alternative first, followed by a new non-terminal representing repeated suffixes.

Left Recursion

A → Aα | B
A → Aα1 | Aα2 | ... | β1 | β2

Remove left Recursion

A → βA'
A' → αA' | ε

3. A → AB | AC | aB | cd
   A → aBA' | cdA'
   A' → BA' | CA' | ε
4. A → ABd | Aa | a

1. A → A + B | B
   A → BA'
   A' → +BA' | ε
2. E → E + T | T
   E → TE'
   E' → +TE' | ε

D. Left factoring

Left factoring removes a common prefix from alternatives.

Original:

$$A \to \alpha\beta_1 \mid \alpha\beta_2$$

Transformed:

$$A \to \alpha A'$$

$$A' \to \beta_1 \mid \beta_2$$

Purpose: Helps a predictive parser decide which production to use with limited lookahead.

E. FIRST and FOLLOW

Consider the grammar:

$$E \to TE'$$

$$E' \to +TE' \mid \varepsilon$$

$$T \to FT'$$

$$T' \to *FT' \mid \varepsilon$$

$$F \to (E) \mid id$$

FIRST sets

Non-terminal	FIRST
E	{(, id}
E′	{+, ε}
T	{(, id}
T′	{*, ε}
F	{(, id}

FOLLOW sets

The dollar sign `$` marks the end of input.

Non-terminal	FOLLOW
E	{), $}
E′	{), $}
T	{+,), $}
T′	{+,), $}
F	{*, +,), $}

FIRST/FOLLOW methodology

- FIRST of a terminal is the terminal itself.
- FIRST of a production is determined by its right-hand side. Continue through nullable symbols when necessary.
- If a non-terminal can derive ε, account for the next symbol in the sequence.
- FOLLOW of the start symbol contains `$`.
- If A → αBβ, add FIRST of β, excluding ε, to FOLLOW(B).
- If β is empty or can derive ε, add FOLLOW(A) to FOLLOW(B).

F. Constructing an LL(1) parsing table

For every production A → α:

1. Find FIRST(α).
2. For every terminal in FIRST(α), place A → α in the appropriate table cell.
3. If ε is in FIRST(α), also place A → α in each column belonging to FOLLOW(A).
4. If a cell needs two different productions, the grammar is not LL(1) under this table construction.

LL(1) table for the grammar above

Non-terminal	`id`	`+`	`*`	`(`	`)`	`$`
E	TE′	Error	Error	TE′	Error	Error
E′	Error	+TE′	Error	Error	ε	ε
T	FT′	Error	Error	FT′	Error	Error
T′	Error	ε	*FT′	Error	ε	ε
F	id	Error	Error	(E)	Error	Error

No cell contains two alternative productions, so this grammar is LL(1).

How to parse using an LL(1) table

- Put `$` and the start symbol on the stack.
- Compare the stack top with the next input token.
- If both are the same terminal, match it and advance the input.
- If the stack top is a non-terminal, look up the table entry using the non-terminal and current input token.
- Replace the non-terminal with the right-hand side of that production, pushing its symbols in reverse order.
- A table error means rejection; successful consumption of input and matching `$` means acceptance.

G. Top-down versus bottom-up parsing

Top-down parsing	Bottom-up parsing
Begins with the start symbol	Begins with input symbols
Builds toward the input string	Reduces the input toward the start symbol
Attempts a leftmost derivation	Uses the reverse of a rightmost derivation in standard shift-reduce parsing
Examples: Recursive descent, predictive LL(1)	Examples: Shift-reduce, LR-family parsers

H. Shift-reduce parsing

Four common actions:

- **Shift:** Move the next input token onto the stack.
- **Reduce:** Replace a handle with the left-hand side of its production.
- **Accept:** Input has been successfully parsed.
- **Error:** The input cannot be handled by the current parsing state and grammar.

A handle is reduced to progressively build the parse toward the grammar's start symbol.

I. Ambiguous grammar

A grammar is ambiguous when at least one string can have more than one distinct parse tree or leftmost/rightmost derivation.

Example:

$$E \to E + E \mid E * E \mid id$$

For `id + id * id`, the grammar does not uniquely specify whether addition or multiplication should be performed first.

Solution concept: Use precedence and associativity rules or rewrite the grammar into separate levels for expressions, terms, and factors.

J. Operator precedence

For the usual arithmetic convention:

- Parentheses have the highest priority.
- Multiplication and division bind more tightly than addition and subtraction.
- Operators of the same precedence generally associate left to right unless defined otherwise.

Example:

$$id + id * id = id + (id * id)$$

The exact precedence rules must come from the grammar or language specification.

K. LEX and YACC quick reference

Tool	Full name / role
LEX	Lexical-analyzer generator
YACC	Yet Another Compiler Compiler. Parser generator
`lex scanner.l`	Generates a C scanner, commonly `lex.yy.c`
`yacc -d parser.y`	Generates parser C source and a token header
`yytext`	Common LEX/Flex variable containing the matched text
`yylex()`	Common lexical-analyzer entry function
`yyparse()`	Common YACC parser entry function
`yyerror()`	Common parser error-handling function

Typical classic UNIX workflow:

```
lex scanner.l
yacc -d parser.y
cc y.tab.c lex.yy.c -ly -ll -o parser
./parser
```

The exact linking flags and generated filenames vary across systems and implementations. Modern Flex/Bison installations may use different commands and library flags.

Other Chapter 6 concepts

- **Regular expression:** Pattern describing a set of strings.
- **DFA:** Deterministic Finite Automaton. A state-machine model commonly used for recognizing regular languages.
- **Recursive descent parser:** A top-down parser implemented through recursive procedures for grammar non-terminals.

- **Predictive parser:** A parser that chooses productions using lookahead without backtracking in the standard LL(1) case.

Chapter 6 quick revision: Left recursion, left factoring, FIRST/FOLLOW, LL(1) table, top-down versus bottom-up, shift-reduce parsing, ambiguity, grammar types, LEX, YACC and DFA.

Chapter 7. Compilers

A. Essential terminology

Term	Definition
Compiler	Translates a source program into another representation, typically producing target code.
Semantic Gap	Gap between the concepts in a high-level problem domain and the low-level operations offered by the machine.
Binding	Association of a program entity with an attribute, such as a name with a type or storage location.
Binding Time	The stage when a particular binding is established.
Scope	The region of a program in which a name is visible or can be referenced under the language's rules.
Activation Record	A memory structure holding information for one procedure/function invocation.
Code Optimization	Improving code efficiency while preserving required behavior.
DAG	Directed Acyclic Graph. A graph useful for representing expression dependencies and shared computations.

B. Binding and binding times

Binding time	Example
Language design time	Meaning and behavior of a language operator are specified
Compile time	A declared variable's type is checked and associated with its declaration
Link time	An external symbol is resolved between object modules
Load time	Relocatable addresses are adjusted for the load location
Run time	A dynamic value or runtime storage location becomes known

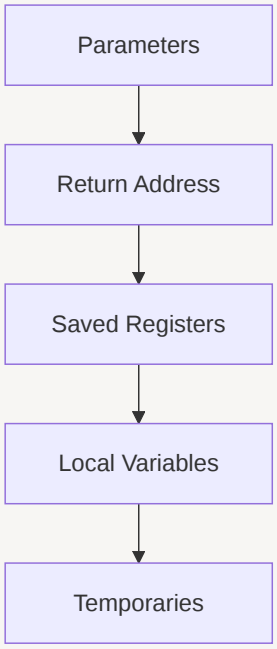
Static binding is determined before execution, according to the language and construct. **Dynamic binding** is determined during execution, often depending on runtime information.

Binding time depends on which particular attribute or association is being discussed.

C. Memory allocation

Type	Meaning	Typical example
Static allocation	Storage determined before execution or fixed for the relevant program lifetime	Fixed global/static data
Stack allocation	Storage allocated and released as procedure calls enter and return	Local variables in ordinary function calls
Heap allocation	Storage allocated and managed dynamically	Dynamically created objects

Activation record




```
+----------------------+
| Parameters           |
+----------------------+
| Return Address       |
+----------------------+
| Saved Registers      |
+----------------------+
| Local Variables      |
+----------------------+
| Temporaries          |
+----------------------+
```

The exact layout depends on the calling convention, compiler, and architecture.

D. Code optimization techniques

Technique	What it does	Example / idea
Constant folding	Evaluates constant expressions earlier	`3 * 4` becomes `12`
Constant propagation	Substitutes known constant values	If `x = 5`, replace a later eligible use of `x` with `5`
Common subexpression elimination	Avoids recomputing an unchanged expression	Reuse an already computed `a + b`
Dead-code elimination	Removes computations that cannot affect observable behavior	Remove an unused temporary assignment
Copy propagation	Replaces copies with the original eligible value	If `x = y`, later uses of `x` may use `y`
Loop-invariant code motion	Moves computations outside a loop if safe	Compute an unchanged expression before the loop

Technique	What it does	Example / idea
Strength reduction	Replaces an expensive operation with a cheaper equivalent when valid	Simplify eligible repeated arithmetic
Peephole optimization	Improves a small sequence of target instructions	Remove a redundant instruction sequence

Optimization must preserve the required program semantics. The validity of a particular transformation can depend on overflow rules, side effects, memory aliasing, and other language details.

E. Triples, quadruples and indirect triples

Consider:

$$x = (a - b) + (c * d)$$

Intermediate steps:

```
t1 = a - b
t2 = c * d
t3 = t1 + t2
x  = t3
```

Quadruple representation

A quadruple stores operator, two operands, and an explicit result.

No.	Operator	Arg1	Arg2	Result
0	`-`	`a`	`b`	`t1`
1	`*`	`c`	`d`	`t2`
2	`+`	`t1`	`t2`	`t3`
3	`:=`	`t3`		`x`

Triple representation

A triple identifies the result by the position of its record, rather than by a separate result-name field.

No.	Operator	Arg1	Arg2
0	`-`	`a`	`b`
1	`*`	`c`	`d`
2	`+`	`(0)`	`(1)`
3	`:=`	`(2)`	`x`

Here, `(0)` refers to the result of triple 0, and similarly for the other indexed references.

Indirect triples

Indirect triples add a pointer or sequence table to identify the order of triple execution. The triples can remain in place while the pointer ordering changes.

Property	Quadruple	Triple	Indirect triple
Explicit result field	Yes	No	No in the underlying triples

Property	Quadruple	Triple	Indirect triple
Reference to previous result	Usually by temporary name	By triple position	By triple position
Reordering	Result names remain explicit	May require reference updates	Pointer ordering can support reordering

F. DAG for common subexpressions

A DAG can represent repeated computations using one shared node.

For example:

```
t1 = a + b
t2 = a + b
```

If neither operand changes between the computations, the compiler may calculate `a + b` once and reuse the value.

This is a common-subexpression optimization opportunity, not an unconditional rule when side effects or changing values are involved.

Chapter 7 quick revision: Binding times, static/dynamic binding, memory allocation, activation records, optimization, DAGs, triples, quadruples and indirect triples.

Chapter 8. Interpreters and Debuggers

A. Essential terminology

Term	Definition
Interpreter	Executes a program or intermediate representation by interpreting instructions.
Interpretation	The process of executing program operations through an interpreter.
JVM	Java Virtual Machine. The abstract execution environment for Java bytecode and compatible class files.
JRE	Java Runtime Environment. A traditional term for the JVM and supporting runtime components.
JDK	Java Development Kit. Includes development tools and the components needed to develop Java programs.
Debugger	A tool that helps inspect and diagnose program execution.
Breakpoint	A point at which execution pauses for inspection.
Stack Trace	A record of the active call sequence associated with an execution point or failure.

B. Compiler versus interpreter

Compiler-oriented execution	Interpreter-oriented execution
Translates the program before the resulting target code is run	Interprets source instructions or an intermediate representation
Can produce a reusable executable	Often relies on an interpreter at execution time
May have a separate compilation stage	May start execution without a traditional ahead-of-time compile-to-machine-code stage

Compiler-oriented execution	Interpreter-oriented execution
Optimization may occur during compilation	Interpretation may be combined with runtime optimizations

Modern runtimes often combine both approaches. For example, an implementation may interpret bytecode and use just-in-time compilation for frequently executed code.

C. Pure versus impure interpreter

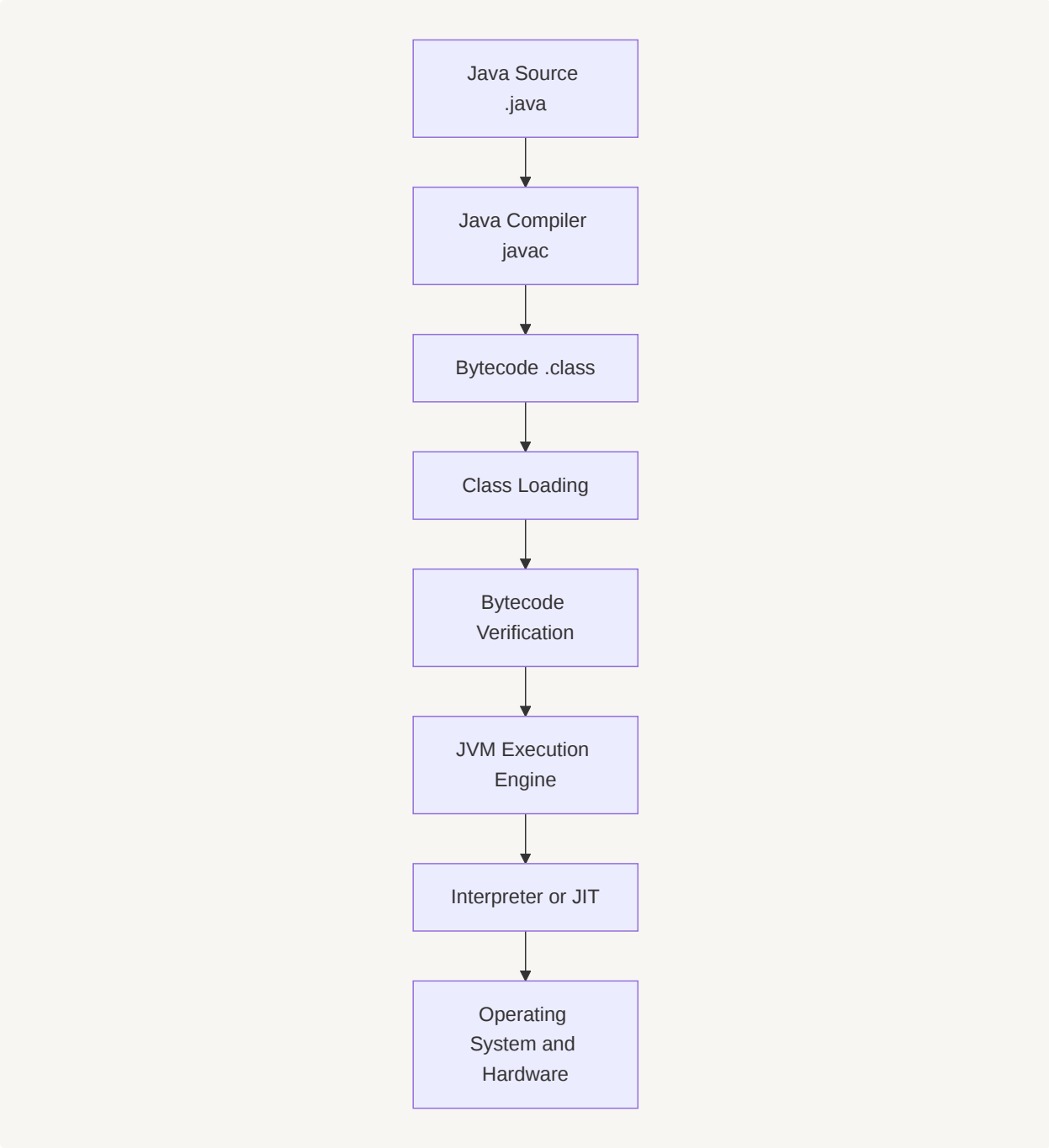
In the conventional distinction used in system-software texts:

- **Pure interpreter:** Directly interprets the source-language program without first translating the whole program into a separate intermediate program for its interpreter.
- **Impure interpreter:** Uses an intermediate representation or translation stage before interpreting that representation.

Terminology and precise boundaries can vary across textbooks. For exam answers, follow the definitions used in the prescribed reference text if it gives a specific distinction.

D. Java Virtual Machine (JVM)

Execution flow



Supporting JVM components include runtime memory areas and garbage collection. Details vary with the JVM implementation.

- **Bytecode:** An intermediate instruction representation stored in Java class files.
- **Class loader:** Loads classes into the JVM.
- **Bytecode verifier:** Checks structural and safety-related properties of bytecode before execution as applicable.
- **Interpreter:** Executes bytecode instructions.

- **JIT compiler:** Just-in-time compiler. Compiles selected code into native machine code at runtime.
- **Garbage collector:** Reclaims heap memory occupied by objects that are no longer reachable, according to the runtime's collection rules.

JVM versus JRE versus JDK

Term	Primary purpose
JVM	Executes bytecode
JRE	Runtime environment and supporting components
JDK	Development kit, including tools such as the Java compiler

E. Types of errors

Error	Meaning	Example
Lexical	Invalid character/token	An illegal character in source code
Syntax	Violates grammar rules	Missing required punctuation
Semantic	Violates language meaning or constraints	Incompatible types in a context requiring compatible values
Runtime	Occurs during execution	Division by zero where the language reports it at runtime
Logical	Program runs but produces incorrect results	Using the wrong formula

An error can be detected at different stages depending on the programming language and implementation.

F. Debugging procedures

1. **Reproduce:** Create the conditions under which the error appears.
2. **Isolate:** Narrow down the component or statements responsible.
3. **Inspect:** Use breakpoints, variable watches, call stacks, and logs.
4. **Diagnose:** Identify the underlying cause.
5. **Correct:** Make the smallest suitable fix.
6. **Test:** Verify the fix and check for regressions.

Common debugger classifications

- **Source-level debugger:** Shows source code, variables, and source-level execution points.
- **Machine-level debugger:** Inspects instructions, registers, memory and machine-level execution.
- **Interactive/dynamic debugger:** Allows inspection while the program is running or paused.
- **Post-mortem debugging:** Examines information such as a core dump or crash record after failure.

These categories are not mutually exclusive; a debugger can belong to multiple categories.

Chapter 8 quick revision: Pure versus impure interpreters, compiler versus interpreter, JVM architecture, JDK/JRE/JVM, error types, debugging procedure and debugger classification.

Chapter 9. Programming Languages

New-syllabus coverage

Your uploaded PYQ analysis identifies this chapter as a **new-syllabus blind spot**, with no past-paper coverage in the analysis period. It should still be prepared because it is explicitly present in the GTU syllabus.

A. Essential terminology

Term	Definition
High-Level Language	A programming language designed to express computation using abstractions that reduce dependence on machine instructions.
Data Type	A classification specifying the kinds of values and permitted operations associated with a value.
Data Structure	A method of organizing and storing data for use.
Storage Allocation	The process of assigning memory storage to program entities.
Scope of Name	The region in which an identifier is visible according to language rules.
Functional Modularity	Dividing a program into functions or modules with clear, focused responsibilities.
Extensibility	The ability to extend a language, program, or system with additional capabilities.
Asynchronous Operation	An operation that can proceed independently without requiring the initiating computation to wait until it finishes.

B. High-level language versus low-level language

High-level language	Low-level language
Uses higher-level abstractions	Closely represents machine instructions or hardware operations
Usually easier to read and maintain	Often requires more hardware-specific knowledge
More portable in many cases	Often more architecture-dependent
Commonly requires a compiler or interpreter	Assembly normally requires an assembler
Hides many hardware details	Exposes more machine-level details

No language is automatically faster or safer solely because it belongs to one category. The language, implementation, compiler, runtime, and program design matter.

C. Common data types

Category	Examples
Integer	Whole numbers
Floating-point	Approximate real-number representation
Character	A character value
Boolean	True/false values
Array	Collection of values accessed by an index
Record/structure	Group of fields that may have different types
Pointer/reference	A value used to refer to another object or storage location, depending on the language

A data type describes values and operations; a data structure describes how data is organized.

D. Storage allocation

Type	Main characteristic
Static	Storage is determined before execution or has a fixed allocation for the relevant lifetime
Stack	Typically follows procedure-call and return lifetimes
Heap	Supports dynamically managed storage

These allocation methods can coexist in one program. The language's memory model determines their exact behavior.

E. Scope of names

- **Lexical/static scope:** The visible binding is determined by the program's source structure.
- **Dynamic scope:** Name resolution depends on the runtime calling context in languages or systems that use dynamic scoping.
- **Local name:** Visible only within its relevant local region.
- **Global name:** Visible in a broader region, subject to declarations and language rules.

Scope and lifetime are different. Scope describes where a name can be referenced; lifetime describes when the associated object or storage exists.

F. Functional modularity

A function or module should ideally have:

- A clearly defined responsibility.
- Well-defined inputs and outputs.
- Minimal unnecessary dependence on other modules.
- A reusable interface.
- Behavior that can be tested independently.

Benefits: Easier testing, debugging, maintenance, reuse, and teamwork.

G. Asynchronous operation

Asynchronous execution allows an initiating task to continue without waiting for another operation to complete.

Examples include:

- Network requests.
- Background file operations.
- Event-driven processing.
- Concurrent or asynchronous task handling.

Asynchronous does not necessarily mean that work happens simultaneously on multiple CPU cores. An event loop can manage asynchronous operations without running every operation in parallel.

H. Extensibility and compile-time considerations

Extensibility concerns how easily a language or program can gain additional capabilities. Examples include adding modules, libraries, language constructs, or plugin functionality.

Compile-time and runtime considerations:

Compile time	Runtime
Translation and static checking	Execution of the translated/interpreted program
Can detect certain type and syntax errors	Can encounter errors dependent on actual execution values
May perform code optimization	May perform dynamic checks and runtime optimization
Can resolve some names and attributes	Can resolve dynamic information where the language/runtime requires it

Chapter 9 quick revision: High-level languages, data types, storage allocation, scope, functional modularity, asynchronous operations, extensibility and compile-time versus runtime considerations.

Final Quick-Reference Sheet

1. Most important full forms

Abbreviation	Full form
SS	System Software
LC	Location Counter
OPTAB	Operation Code Table
SYMTAB	Symbol Table
LITTAB	Literal Table
POOLTAB	Pool Table
MNT	Macro Name Table
MDT	Macro Definition Table
ALA	Argument List Array
PNTAB	Parameter Name Table
KPDTAB	Keyword Parameter Default Table
EVTAB	Expansion Variable Table
SSTAB	Sequence Symbol Table
LL(1)	Left-to-right scan, leftmost derivation, one lookahead token
LEX	Lexical-analyzer generator
YACC	Yet Another Compiler Compiler
DAG	Directed Acyclic Graph
JVM	Java Virtual Machine
JRE	Java Runtime Environment
JDK	Java Development Kit
JIT	Just-in-Time compilation/compiler
IR	Intermediate Representation
DFA	Deterministic Finite Automaton

2. High-priority differences

Pair	Core difference
Compiler vs Interpreter	Translation into another representation versus execution through interpretation
Assembler vs Compiler	Assembly-to-machine/object code versus translation from a higher-level language
Linker vs Loader	Combines modules and resolves symbols versus places a program in memory
One-pass vs Two-pass assembler	Single principal scan versus separate address/table and code-generation passes
Macro vs Subroutine	Expansion during processing versus runtime call and return
Top-down vs Bottom-up parser	Start symbol toward input versus input toward start symbol
Triple vs Quadruple	Result identified by record position versus explicit result field
Static vs Dynamic binding	Binding established before execution versus at runtime
Static vs Stack vs Heap allocation	Fixed allocation model versus call-lifetime allocation versus dynamic storage

3. Assembly directives at a glance

Directive	Remember this
`START`	Set initial LC
`END`	Finish assembly
`ORIGIN`	Change LC
`EQU`	Assign a value to a symbol
`LTORG`	Assign addresses to pending literals
`DS`	Reserve storage
`DC`	Define and initialize a constant

4. Parsing problem-solving checklist

For a grammar-based numerical question:

1. Write the original productions neatly.
2. Check and remove immediate left recursion if required.
3. Apply left factoring if needed.
4. Calculate FIRST sets.
5. Calculate FOLLOW sets.
6. Fill the LL(1) table.
7. Check for multiple entries in any cell.
8. Trace the requested input using the stack and input pointer.
9. State acceptance or rejection.

5. Assembler numerical checklist

For a symbol-table/literal-table question:

1. Initialize LC from `START`.
2. Assign each instruction's address.
3. Update LC after each instruction or allocated storage unit.

4. Record labels in SYMTAB.
5. Record literals in LITTAB.
6. Process `EQU` without allocating storage for the symbol itself.
7. Apply `ORIGIN` to reset LC.
8. Assign pending literal addresses at `LTORG` or the applicable end-of-program processing.
9. Build POOLTAB according to the indexing convention.
10. Resolve forward references and show the final intermediate/object-code representation required by the question.

6. GTU practical connection

Your uploaded syllabus lists practical work covering:

- Lexical-analyzer implementation and the LEX utility.
- Left factoring and left-recursion removal.
- Recursive-descent parsing.
- Predictive parsing.
- SAL programs that generate SYMTAB and LITTAB.
- C macro features.
- Quadruple-table generation.
- String parsing using a predictive parser.

These practicals reinforce Chapters 3, 4, 6 and 7, especially the grammar transformations, symbol tables, and intermediate-code generation.

> **Compiled BY** : Manthan A. D. [in]